430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Chapter 3: Functions

Chapter 3
: Learning objectives

- Predefined Functions
  - Functions that are already implemented in C++
  - Those that return a value and those that don't
- Programmer-defined Functions
  - Functions that can be defined by a programmer
  - Defining, Declaring, Calling
  - Recursive Functions
- Scope Rules
  - Local variables
  - Global constants and global variables
  - Blocks, nested scopes

Introduction to Functions

- Building Blocks of Programs
- Other terminology in other languages:
  - Procedures, subprograms, methods (Java)
  - In C++ → functions
- I-P-O
  - Input – Process – Output
  - Basic subparts to any program
  - Use functions for these "pieces"

Predefined Functions

- Library files contain many predefined functions
- Two types:
  - Those that return a value
  - Those that do not (void)
- To use a predefined function, you must "#include" appropriate library
  - E.g.,
    - `<cmath>, <cstdlib>` (Original "C" libraries)
    - `<iostream>` (for cout, cin)

Using Predefined Functions

- Math functions
  - Found in library `<cmath>`
  - Most return a value (the "answer")
- E.g., `theRoot = sqrt(9.0);`
  - Components:

    `sqrt` = name of a library function
    `theRoot` = variable used to assign "answer" to
    `9.0` = argument or "starting input" for function
  - In I-P-O:
    - I = 9.0
    - P = "compute the square root"
    - O = 3, which is returned & assigned to `theRoot`

The Function Call

- Back to this assignment:

```
theRoot = sqrt(9.0);
```

  - The expression "`sqrt(9.0)`" is known as a function *call*, or function *invocation*
  - The argument in a function call `(9.0)` can be a literal, a variable, or an expression, i.e., followings all allowed

```
sqrt(9.0), sqrt(a), sqrt(a*b)
```

  - The call itself can be part of an expression:
    - `bonus = sqrt(sales)/10;`
    - A function call is allowed wherever it's legal to use an expression of the function's return type

A Larger Example:
Display 3.1 A Predefined Function That Returns a Value (1 of 2)

Display 3.1 **A Predefined Function That Returns a Value**

```
1  //Computes the size of a doghouse that can be purchased
2  //given the user's budget.
3  #include <iostream>
4  #include <cmath>
5  using namespace std;

6  int main( )
7  {
8      const double COST_PER_SQ_FT = 10.50;
9      double budget, area, lengthSide;

10     cout << "Enter the amount budgeted for your doghouse $";
11     cin >> budget;

12     area = budget / COST_PER_SQ_FT;
13     lengthSide = sqrt(area);
```

A Larger Example:
Display 3.1 A Predefined Function That Returns a Value (2 of 2)

```
14      cout.setf(ios::fixed);
15      cout.setf(ios::showpoint);
16      cout.precision(2);
17          cout << "For a price of $" << budget << endl
18            << "I can build you a luxurious square doghouse\n"
19            << "that is " << lengthSide
20            << " feet on each side.\n";

21      return 0;
22  }
```

Sets options for printing decimal point

Sample Dialogue

```
Enter the amount budgeted for your doghouse $25.00
For a price of $25.00
I can build you a luxurious square doghouse
that is 1.54 feet on each side.
```

More Predefined Functions

- `#include <cstdlib>`
  - A library containing functions like:
    - `abs()` // Returns absolute value of an int
    - `fabs()` // Returns absolute value of a float
  - `fabs()` is actually also in a library `<cmath>`
  - You must include at least one library containing the function. (It's fine to include two or more such libraries)
    - You can refer to manuals/help to find out a library containing a specific function

More Math Functions

- `pow(x, y)`
  - Returns `x` to the power `y`:

```
double answer, x = 3.0, y = 2.0;
answer = pow(x, y);
cout << answer;
```

    - Here `9.0` is displayed since $3.0^{2.0} = 9.0$

- Notice this function receives **two** arguments
  - In fact, a function can have **any finite** number of arguments of varying data types
  - No function has more than one value returned! (different from Python)

Even More Math Functions:
Display 3.2 Some Predefined Functions (1 of 2)

Display 3.2 Some Predefined Functions

All these predefined functions require `using namespace std;` as well as an `include` directive.

NAME	DESCRIPTION	TYPE OF ARGUMENTS	TYPE OF VALUE RETURNED	EXAMPLE	VALUE	LIBRARY HEADER
`sqrt`	Square root	`double`	`double`	`sqrt(4.0)`	`2.0`	`cmath`
`pow`	Powers	`double`	`double`	`pow(2.0,3.0)`	`8.0`	`cmath`
`abs`	Absolute value for `int`	`int`	`int`	`abs(-7)` `abs(7)`	`7` `7`	`cstdlib`
`labs`	Absolute value for `long`	`long`	`long`	`labs(-70000)` `labs(70000)`	`70000` `70000`	`cstdlib`
`fabs`	Absolute value for `double`	`double`	`double`	`fabs(-7.5)` `fabs(7.5)`	`7.5` `7.5`	`cmath`

Even More Math Functions:
Display 3.2 Some Predefined Functions (2 of 2)

`ceil`	Ceiling (round up)	`double`	`double`	`ceil(3.2)` `ceil(3.9)`	`4.0` `4.0`	`cmath`
`floor`	Floor (round down)	`double`	`double`	`floor(3.2)` `floor(3.9)`	`3.0` `3.0`	`cmath`
`exit`	End program	`int`	`void`	`exit(1);`	None	`cstdlib`
`rand`	Random number	None	`int`	`rand( )`	Varies	`cstdlib`
`srand`	Set seed for `rand`	`unsigned int`	`void`	`srand(42);`	None	`cstdlib`

Predefined Void Functions

- A function with **no** returned value
- Performs an action, but sends no "answer"
- When called, it's a statement itself
  - Ex: `exit(1);` // No return value, so not assigned
    - This call terminates program
    - void functions can still have arguments
- All aspects are the same as functions that "return a value"
  - They just don't return a value!

Random Number Generator

- Returns a "randomly chosen" number
- Used for simulations, games
  - `rand()`
    - Takes no arguments
    - Returns a uniform random integer value between 0 & RAND_MAX (= a very large integer constant)
  - Scaling
    - Squeezes random number into smaller range Ex: `rand()%6`
    - Returns random value between 0 & 5
  - Shifting
    - Ex: `rand()%6 + 1`
    - Shifts range between 1 & 6 (e.g., die roll)
  - Real number in [0.0 1.0] (typecasting happening)
    - `(double)(RAND_MAX - rand())/ RAND_MAX`

Random Number Seed

- Pseudorandom numbers
  - Calls to `rand()` produce a fixed "sequence" of random numbers
- Use "seed" to alter sequence `srand(seed_value);`
  - It is a void function
  - Receives one argument, the "seed"
  - Can use any seed value, including system time: `srand(time(0));`
  - `time(0)` returns system time as numeric value
  - Library `<time>` contains `time()` functions

```
int i;
srand(99);
for (i = 0; i < 10; i++)
    cout << (rand( ) % 11) << endl;
srand(99);
for (i = 0; i < 10; i++)
    cout << (rand( ) % 11) << endl;
```

Programmer-Defined Functions

- Write your own functions!

- Building blocks of programs
  - Divide & Conquer
  - Readability
  - Re-use

- Your "definition" can go in either:
  - Same file as the main() function
  - Separate file so others can use it, too

Components of Function Use

- Three pieces for using functions:
  - **Function Declaration/Prototype**
    - Information for compiler
    - To properly interpret calls
  - **Function Definition**
    - Actual implementation/code for what a function does
  - **Function Call**
    - Transfer control to function

Function Declaration/Prototype

- An "informational" declaration for compiler
- Tells compiler how to interpret calls
  - Syntax:
    `<return_type> FcnName(<formal-parameter-list>);`
  - Example:

Notice the semicolon!

```
double totalCost(int numberParameter, double priceParameter);
```

  - Designates the type of the return value, name of the function, number of arguments, type of the formal parameters

- Placed **before** any calls to the function
  - **Above** the `main()` function in global space

Alternative Function Declaration/Prototype

- Compiler only needs to know:
  - Return type
  - Function name
  - Parameter list
- Formal parameter names are not needed:

```
double totalCost(int, double);
```

- Can only present the data types of the parameters

Function Definition

- Actual implementation of function
  - Just like implementing the function `main()`
- Example:

```
double totalCost(int numberParameter, double priceParameter)
{
    const double TAXRATE = 0.05; //5% sales tax
    double subtotal;

    subtotal = priceParameter * numberParameter;
    return (subtotal + subtotal*TAXRATE);
}
```

Function body

Function definition

- Notice proper indenting

Function Definition Placement

- Can be placed ***after*** or ***before*** the function `main()`
  - In case of **before**, function declaration can be omitted
  - **NOT** "inside" the `main()` function!

- Can also be placed in a separate file
  - The separate file can be named with **.cpp** extension
  - The function declaration can be placed in another header file

    `#include "XXX.h" (or "XXX.hpp")`
  - Examples are given later

Function Definition Placement

- Functions are "equal"; no function is "part" of another
- Parameters in the definition
  - Called as the **formal parameters** and must be included ⇔ function declaration
  - "**Memory Placeholders**" for data sent in
    - "Variable name" used to refer to the data in the definition part
- Return statement
  - Sends data back to the caller

Function Call

- Just like calling a predefined function

```
bill=totalCost(number, price);
```

- Recall: `totalCost` returns a double value
  - Assigned to variable named "bill"
- **Arguments** here: `number,price`
  - Arguments can be literals, variables, expressions, or their combination
  - In a function call, arguments often called as "actual arguments"
    - Because they contain the "actual data" being sent

A simple example: Order matters

```
int volume(int h, int w, int l) { return h*w*l; }
int area(int h, int w, int l) { return 2*(h*w + w*l + l*h); }

void main() {
   std::cout << "The volume of box1 is “<< volume(3,5,7) << std::endl;
   std::cout << "The area of box1 is " << area(3,5,7) << std::endl;
   std::cout << "The area of box2 is " << area(10,20,30) << std::endl;
   std::cout << "The area of box3 is " << area(15,25,35) << std::endl;
}
```

OK

```
void main() {
   std::cout << "The volume of box1 is “<< volume(3,5,7) << std::endl;
   std::cout << "The area of box1 is " << area(3,5,7) << std::endl;
   std::cout << "The area of box2 is " << area(10,20,30) << std::endl;
   std::cout << "The area of box3 is " << area(15,25,35) << std::endl;
}

int volume(int h, int w, int l) { return h*w*l; }
int area(int h, int w, int l) { return 2*(h*w + w*l + l*h); }
```

error

A simple example: Function declaration/prototypes

```
#include <iostream>

int volume(int h, int w, int l);
int area(int h, int w, int l);

void main() {
   std::cout << "The volume of box1 is "<< volume(3,5,7) << std::endl;
   std::cout << "The area of box1 is " << area(3,5,7) << std::endl;
   std::cout << "The area of box2 is " << area(10,20,30) << std::endl;
   std::cout << "The area of box3 is " << area(15,25,35) << std::endl;
}

int volume(int h, int w, int l) { return h*w*l; }
int area(int h, int w, int l) { return 2*(h*w + w*l + l*h); }
```

function declarations/prototypes

OK

A simple example: Function prototypes

```
#include <iostream>

int volume(int, int, int );
int area(int, int, int );

void main() {
   std::cout << "The volume of box1 is "<< volume(3,5,7) << std::endl;
   std::cout << "The area of box1 is " << area(3,5,7) << std::endl;
   std::cout << "The area of box2 is " << area(10,20,30) << std::endl;
   std::cout << "The area of box3 is " << area(15,25,35) << std::endl;
}

int volume(int h, int w, int l) { return h*w*l; }
int area(int h, int w, int l) { return 2*(h*w + w*l + l*h); }
```

function declarations/prototypes
(parameter names can be omitted)

OK

A simple example: Function prototypes

A.cpp

modular programming!

```
#include <iostream>

int volume(int h, int w, int l);
int area(int h, int w, int l);

void main() {
   std::cout << "The volume of box1 is "<< volume(3,5,7) << std::endl;
   std::cout << "The area of box1 is " << area(3,5,7) << std::endl;
   std::cout << "The area of box2 is " << area(10,20,30) << std::endl;
   std::cout << "The area of box3 is " << area(15,25,35) << std::endl;
}
```

B.cpp

```
int volume(int h, int w, int l) { return h*w*l; }
int area(int h, int w, int l) { return 2*(h*w + w*l + l*h); }
```

OK

Need to compile two files together!

```
g++ A.cpp B.cpp -o sample
```

A simple example: Most recommended way

myheader.h

```
int volume(int h, int w, int l);
int area(int h, int w, int l);
// and other function prototypes...
```

A.cpp

```
#include <iostream>
#include "myheader.h"

void main() {
   std::cout << "The volume of box1 is "<< volume(3,5,7) << std::endl;
   std::cout << "The area of box1 is " << area(3,5,7) << std::endl;
   std::cout << "The area of box2 is " << area(10,20,30) << std::endl;
   std::cout << "The area of box3 is " << area(15,25,35) << std::endl;
}
```

B.cpp

```
#include "myheader.h" ← Can omit.
int volume(int h, int w, int l) { return h*w*l; }
int area(int h, int w, int l) { return 2*(h*w + w*l + l*h); }
```

OK

More Function Example:
Display 3.5 A Function to Calculate Total Cost (1 of 2)

Display 3.5 A Function Using a Random Number Generator

```
1  #include <iostream>
2  using namespace std;

3  double totalCost(int numberParameter, double priceParameter);
4  //Computes the total cost, including 5% sales tax,
5  //on numberParameter items at a cost of priceParameter each.

6  int main( )
7  {
8      double price, bill;
9      int number;

10     cout << "Enter the number of items purchased: ";
11     cin >> number;
12     cout << "Enter the price per item $";
13     cin >> price;

14     bill = totalCost(number, price);
```

Function declaration; also called the function prototype

Function call

More Function Example:
Display 3.5 A Function to Calculate Total Cost (1 of 2)

```
15      cout.setf(ios::fixed);
16      cout.setf(ios::showpoint);
17      cout.precision(2);
18      cout << number << " items at "
19           << "$" << price << " each.\n"
20           << "Final bill, including tax, is $" << bill
21           << endl;

22      return 0;
23  }

24  double totalCost(int numberParameter, double priceParameter)
25  {
26      const double TAXRATE = 0.05; //5% sales tax
27      double subtotal;

28      subtotal = priceParameter * numberParameter;
29      return (subtotal + subtotal*TAXRATE);
30  }
```

Function head

Function definition

Function body

Sample Dialogue

```
Enter the number of items purchased: 2
Enter the price per item: $10.10
2 items at  $10.10 each.
Final bill, including tax, is $21.21
```

Parameter vs. Argument

- Terms often used interchangeably
- Formal parameters/arguments
  - In function declaration
  - In function definition's header
- Actual parameters/arguments
  - In function call
- Technically, parameter is "formal" piece while argument is "actual" piece

Functions Calling Functions

- We're already doing this!
  - `main()` IS a function!
- Only requirement:
  - Function's declaration must appear first
- Function's definition is typically elsewhere
  - After or before `main()`'s definition
  - Or in a separate file
- Common for functions to call many other functions
- Function can even call itself → "Recursion"

Functions Calling Functions
: Display 3.6 `round` function (1 of 2)

Display 3.6 The Function `round` (part 1 of 2)

```
1  #include <iostream>
2  #include <cmath>
3  using namespace std;

4  int round(double number);
5  //Assumes number >= 0.
6  //Returns number rounded to the nearest integer.
7  int main( )
8  {
9      double doubleValue;
10     char ans;
```

Testing program for the function `round`

Functions Calling Functions : **Display 3.6** `round` function (2 of 2)

Display 3.6 The Function `round` (part 2 of 2)

```
11      do
12      {
13          cout << "Enter a double value: ";
14          cin >> doubleValue;
15          cout << "Rounded that number is " << round(doubleValue) <<
            endl;
16          cout << "Again? (y/n): ";
17          cin >> ans;
18      } while (ans == 'y' || ans == 'Y');
19      cout << "End of testing.\n";

20      return 0;
21  }
22  //Uses cmath:
23  int round(double number)
24  {
25      return static_cast<int>(floor(number + 0.5));
26  }
```

Function calling another function!

Sample Dialogue

```
Enter a double value: 9.6
Rounded, that number is 10
Again? (y/n): y
Enter a double value: 2.49
Rounded, that number is 2
Again? (y/n): n
End of testing.
```

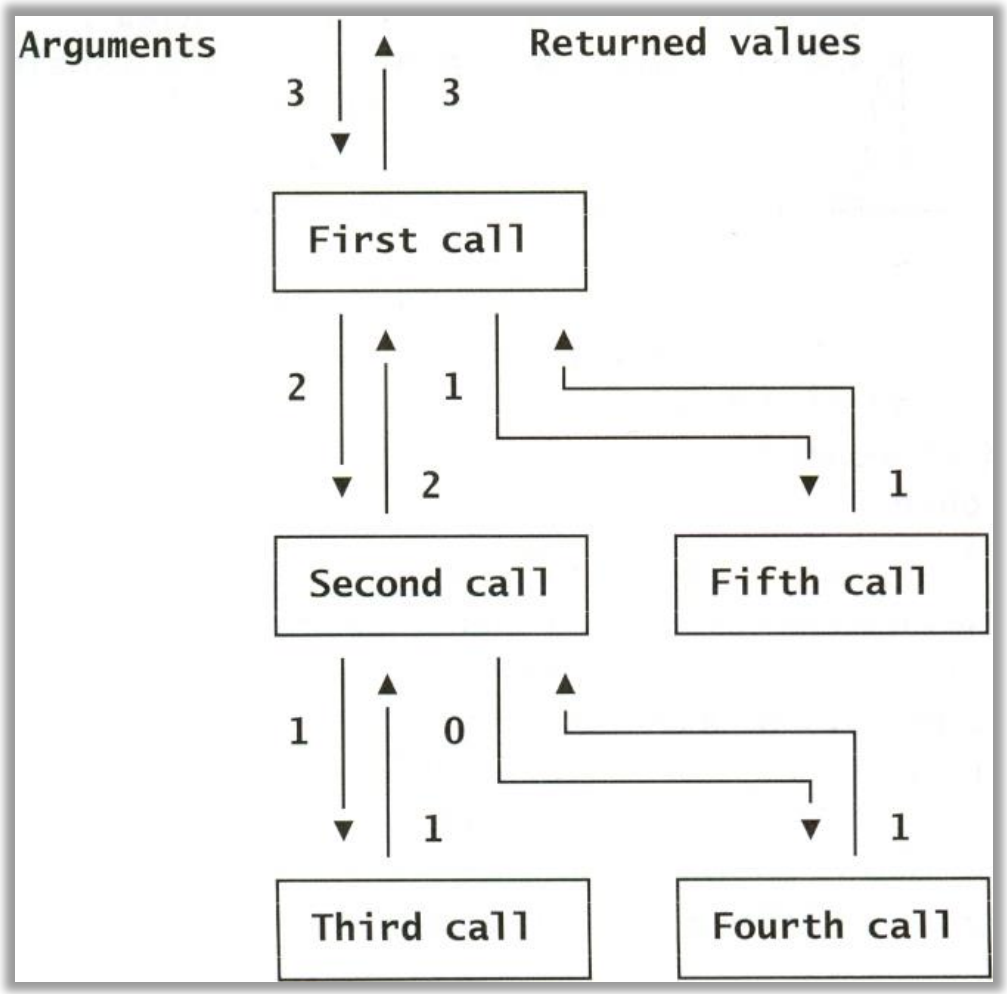
Functions Calling Functions
: Recursive function

```
#include <iostream>

int f(int n) {
    if(n==0 || n==1)
        return 1;
    else
        return f(n-1) + f(n-2);
}

void main() {
    std::cout << f(3) << std::endl;
    std::cout << f(10) << std::endl;
}
```

Functions Calling Functions
: Recursive function



More in Ch.13 and Project 1!

Boolean Return-Type Functions

- Return-type can be any valid type, including Boolean
  - Consider the Boolean expression in

```
if (((rate > = 10) && (rate < 20)) || (rate == 0))
{
      …
}
```

Looks complicated & hard to read

  - We can utilize function to make it look simpler

```
if (appropriate(rate))
{
      …
}
```

```
bool appropriate(int rate)
{
    return (((rate >= 10) && (rate < 20)) || (rate == 0));
}
```

Declaring Void Functions

- Similar to functions returning a value
- Return type specified as "void"
- Example:
  - Function declaration/prototype:

```
void showResults(double fDegrees, double cDegrees);
```

    - Return-type is "void"
    - Nothing is returned

Declaring Void Functions

– Function definition

```
void showResults(double fDegrees, double cDegrees)
{
    cout.setf(ios::fixed);
    cout.setf(ios::showpoint);
    cout.precision(1);
    cout << fDegrees
         << " degrees Fahrenheit is equivalent to\n"
         << cDegrees << " degrees Celsius.\n";
}
```

- Notice: **no** return statement

Calling Void Functions

- Same as calling predefined void functions
- From some other function, like main():
  - `showResults(degreesF, degreesC);`
  - `showResults(32.5, 0.3);`
  - Notice no assignment to other variable, since no value is returned
- Actual arguments (`degreesF, degreesC`)
  - Passed to the function
  - Function is called to "do its job" with the data passed in
- Void function with ***no argument*** is perfectly fine!

```
void initializeScreen( )
{
    cout << endl;
}
```

More on Return Statements

- Transfers control back to the "calling" function
  - Return type other than `void` MUST have return statement
  - Typically, the LAST statement in a function definition

- `return;` is an optional statement for void functions
  - Closing } would implicitly return control from `void` function

Preconditions and Postconditions

- A good way for ***commenting*** a function declaration
- Similar to "I-P-O" discussion
- Comment function declaration
  to describe the input and output of the function :

```
void showInterest(double balance, double rate);
//Precondition: balance is a nonnegative savings account balance.
//rate is the interest rate expressed as a percentage, such as 5
//for 5%.
//Postcondition: The amount of interest on the given balance
//at the given rate is shown on the screen.
```

- Often called Inputs & Outputs

`main()`: "Special"

- Recall: `main()` IS a function
- "Special" in that:
  - One and only one function called `main()` will exist in a program
- Who calls `main()`?
  - Operating system
  - Tradition holds it should have return statement
    - Value returned to "caller" → Here: operating system
  - Should return "`int`" or "`void`"

Scope Rules

- Local variables
  - Declared inside the body of given function
  - Available only within that function
- Can have variables with same names declared in different functions
  - Scope is local: "that function is its scope"
- Local variables preferred
  - Maintain individual control over data
  - Functions should declare whatever local data needed to "do their job"

Local variable example

Display 3.8 **Local Variables** (part 1 of 2)

```
1  //Computes the average yield on an experimental pea growing patch.
2  #include <iostream>
3  using namespace std;

4  double estimateOfTotal(int minPeas, int maxPeas, int podCount);
5  //Returns an estimate of the total number of peas harvested.
6  //The formal parameter podCount is the number of pods.
7  //The formal parameters minPeas and maxPeas are the minimum
8  //and maximum number of peas in a pod.

9  int main( )
10 {
11     int maxCount, minCount, podCount;
12     double averagePea, yield;

13     cout << "Enter minimum and maximum number of peas in a pod: ";
14     cin >> minCount >> maxCount;
15     cout << "Enter the number of pods: ";
16     cin >> podCount;
17     cout << "Enter the weight of an average pea (in ounces): ";
18     cin >> averagePea;

19     yield =
20           estimateOfTotal(minCount, maxCount, podCount) * averagePea;

21     cout.setf(ios::fixed);
22     cout.setf(ios::showpoint);
23     cout.precision(3);
24     cout << "Min number of peas per pod = " << minCount << endl
25          << "Max number of peas per pod = " << maxCount << endl
26          << "Pod count = " << podCount << endl
27          << "Average pea weight = "
28          << averagePea << " ounces" << endl
29          << "Estimated average yield = " << yield << " ounces"
30          << endl;

31     return 0;
32 }
```

*This variable named **averagePea** is local to the **main** function.*

```
34 double estimateOfTotal(int minPeas, int maxPeas, int podCount)
35 {
36     double averagePea;

37     averagePea = (maxPeas + minPeas)/2.0;
38     return (podCount * averagePea);
39 }
```

*This variable named **averagePea** is local to the function **estimateOfTotal**.*

Display 3.8 **Local Variables** (part 2 of 2)

Sample Dialogue

```
Enter minimum and maximum number of peas in a pod: 4 6
Enter the number of pods: 10
Enter the weight of an average pea (in ounces): 0.5
Min number of peas per pod = 4
Max number of peas per pod = 6
Pod count = 10
Average pea weight = 0.500 ounces
Estimated average yield = 25.000 ounces
```

Procedural Abstraction

- Need to know "what" function does, not "how" the function does it!
- Think it as a "black box"
  - Device you know how to use, but not its method of operation
- Implement a function like a black box
  - User of a function only needs to know : declaration
  - Does NOT need to know function definition
    - Also called as ***information hiding***
    - Hide details of "how" function does its job

Global Constants and Global Variables

- If declared "outside" function body
  - Global to all functions in that file
- If declared "inside" function body
  - Local to that function
- Global declarations are typical for constants:
  - `const double TAXRATE = 0.05;`
  - Declare globally so all functions can use it
- Global variables?
  - Possible, but SELDOM-USED
  - Dangerous: no control over usage!

Blocks

- Declare data inside compound statement {}
  - Called a "block"
  - Has "block-scope"
- Note: all function definitions are blocks!
  - This provides local "function-scope"
- Loop block example:

```
for (int n = 1; n <= 10; n++)
        sum = sum + n;
```

  - Variable `n` has scope in loop body block only

Summary I

- Two kinds of functions:
  - "Return-a-value" and void functions
- Functions should be "black boxes"
  - Hide "how" details
  - Declare own local data
- Function declarations should self-document
  - Provide pre- & post-conditions in comments
  - Provide all "caller's" needs for use

Summary 2

- Local data
  - Declared in function definition
- Global data
  - Declared above function definitions
  - OK for constants, not for variables
- Parameters/Arguments
  - Formal: In function declaration and definition
    - Placeholder for incoming data
  - Actual: In function call
    - Actual data passed to function

Thank you!

Web: http://mindlab-snu.github.io
E-mail: tsmoon@snu.ac.kr